

Semana 6 - Algoritmos de Otimização

João Florindo

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas - Brasil
florindo@unicamp.br

Outline

- 1 Gradiente Descendente por Mini-Batch
- 2 Média Ponderada Exponencialmente
- 3 Métodos Avançados de Otimização
- 4 O Problema dos Ótimos Locais
- 5 Juntando as Peças

- Algoritmos avançados de otimização aceleram bastante o treinamento de redes neurais profundas
- Processo do *deep learning* é altamente empírico e o modelo precisa ser treinado várias vezes com muitos dados
- Vetorização: todos os exemplos ao mesmo tempo em matrizes (X, Y)
- Mas e se m for muito grande, e.g., $m = 5$ milhões?
- Passar por todos os exemplos em cada época é muito custoso

- Algoritmos avançados de otimização aceleram bastante o treinamento de redes neurais profundas
- Processo do *deep learning* é altamente empírico e o modelo precisa ser treinado várias vezes com muitos dados
- Vetorização: todos os exemplos ao mesmo tempo em matrizes (X, Y)
- Mas e se m for muito grande, e.g., $m = 5$ milhões?
- Passar por todos os exemplos em cada época é muito custoso

- Algoritmos avançados de otimização aceleram bastante o treinamento de redes neurais profundas
- Processo do *deep learning* é altamente empírico e o modelo precisa ser treinado várias vezes com muitos dados
- Vetorização: todos os exemplos ao mesmo tempo em matrizes (X, Y)
- Mas e se m for muito grande, e.g., $m = 5$ milhões?
- Passar por todos os exemplos em cada época é muito custoso

- Algoritmos avançados de otimização aceleram bastante o treinamento de redes neurais profundas
- Processo do *deep learning* é altamente empírico e o modelo precisa ser treinado várias vezes com muitos dados
- Vetorização: todos os exemplos ao mesmo tempo em matrizes (X, Y)
- Mas e se m for muito grande, e.g., $m = 5$ milhões?
- Passar por todos os exemplos em cada época é muito custoso

- Algoritmos avançados de otimização aceleram bastante o treinamento de redes neurais profundas
- Processo do *deep learning* é altamente empírico e o modelo precisa ser treinado várias vezes com muitos dados
- Vetorização: todos os exemplos ao mesmo tempo em matrizes (X, Y)
- Mas e se m for muito grande, e.g., $m = 5$ milhões?
- Passar por todos os exemplos em cada época é muito custoso

- SOLUÇÃO: Dividir em subconjuntos bem menores, e.g., 1000 exemplos cada (*mini-batch*)
- Representamos cada *mini-batch* por um índice sobrescrito entre chaves

$$\underline{X} = \begin{bmatrix} \underbrace{x^{(1)} \quad x^{(2)} \quad x^{(3)} \quad \dots \quad x^{(1000)}}_{X^{\{1\}} \quad (n_x, 1000)} \mid \underbrace{x^{(1001)} \quad \dots \quad x^{(2000)}}_{X^{\{2\}} \quad (n_x, 1000)} \mid \dots \mid \underbrace{\dots \quad x^{(m)}}_{X^{\{5,000\}} \quad (n_x, 1000)} \end{bmatrix}$$

(n_x, m)

$$\underline{Y} = \begin{bmatrix} \underbrace{y^{(1)} \quad y^{(2)} \quad y^{(3)} \quad \dots \quad y^{(1000)}}_{Y^{\{1\}} \quad (1, 1000)} \mid \underbrace{y^{(1001)} \quad \dots \quad y^{(2000)}}_{Y^{\{2\}} \quad (1, 1000)} \mid \dots \mid \underbrace{\dots \quad y^{(m)}}_{Y^{\{5,000\}} \quad (1, 1000)} \end{bmatrix}$$

(1, m)

What if $m = 5,000,000$?

5,000 mini-batches of 1,000 each

Mini-batch t : $\underline{X^{\{t\}}}, \underline{Y^{\{t\}}}$

Relembrando a notação:

- (i) : i -ésimo exemplo de treino
- $[l]$: l -ésima camada da rede neural
- $\{t\}$: t -ésimo *mini-batch*

Gradiente Descendente com Mini-batch

Mesmo passo vetorizado de antes para todo o conjunto X, Y , mas agora para cada *mini-batch* $X^{t\{t\}}, Y^{t\{t\}}$

Mini-batch gradient descent

Repeat $\sum$
for $t = 1, \dots, 5000$ $\{$

Forward prop on $X^{t\{t\}}$.

$$Z^{t\{t\}} = W^{t\{t\}} X^{t\{t\}} + b^{t\{t\}}$$

$$A^{t\{t\}} = \sigma^{t\{t\}}(Z^{t\{t\}})$$

$\vdots$

$$A^{t\{t\}} = \sigma^{t\{t\}}(Z^{t\{t\}})$$

} Vectorized implementation
(1000 examples)

for $X^{t\{t\}}, Y^{t\{t\}}$

Compute cost $J = \frac{1}{1000} \sum_{i=1}^L \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_{\mathbf{a}} \|W^{t\{t\}}\|_F^2$

Backprop to compute gradients w.r.t $J^{t\{t\}}$ (using $(X^{t\{t\}}, Y^{t\{t\}})$)

$$W^{t\{t\}} := W^{t\{t\}} - \alpha dW^{t\{t\}}, \quad b^{t\{t\}} := b^{t\{t\}} - \alpha db^{t\{t\}}$$

}
}

"1 epoch"

pass through training set.

1 step of gradient descent
using $X^{t\{t\}}, Y^{t\{t\}}$.
(as if $m=1000$)

X, Y

NOTA: Chamamos cada passada pelo conjunto inteiro de **uma época**

Gradiente Descendente com Mini-batch

Mesmo passo vetorizado de antes para todo o conjunto X, Y , mas agora para cada *mini-batch* $X^{t\{t\}}, Y^{t\{t\}}$

Mini-batch gradient descent

Repeat $\sum$
for $t = 1, \dots, 5000$

Forward prop on $X^{t\{t\}}$.

$$Z^{t\{t\}} = W^{t\{t\}} X^{t\{t\}} + b^{t\{t\}}$$

$$A^{t\{t\}} = \sigma^{t\{t\}}(Z^{t\{t\}})$$

$\vdots$

$$A^{t\{t\}} = \sigma^{t\{t\}}(Z^{t\{t\}})$$

} Vectorized implementation
(1000 examples)

Compute cost $J = \frac{1}{1000} \sum_{i=1}^L \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2 \cdot 1000} \sum_{i=1}^L \|W^{t\{t\}}\|_F^2$

Backprop to compute gradients w.r.t $J^{t\{t\}}$ (using $X^{t\{t\}}, Y^{t\{t\}}$)

$$W^{t\{t\}} := W^{t\{t\}} - \alpha dW^{t\{t\}}, \quad b^{t\{t\}} := b^{t\{t\}} - \alpha db^{t\{t\}}$$

}
}

"1 epoch"

pass through training set.

1 step of gradient descent
using $X^{t\{t\}}, Y^{t\{t\}}$.
(as if $m=1000$)

X, Y

NOTA: Chamamos cada passada pelo conjunto inteiro de **uma época**

Entendendo Melhor o Gradiente Descendente em Mini-batch

- No treinamento com todos os exemplos (*batch*), espera-se que o custo diminua em cada iteração
- No *mini-batch* a curva é muito mais ruidosa, mas ainda com tendência de queda
- Um *mini-batch* pode ser mais fácil, o seguinte mais difícil, etc., seguindo uma sequência aleatória

Entendendo Melhor o Gradiente Descendente em Mini-batch

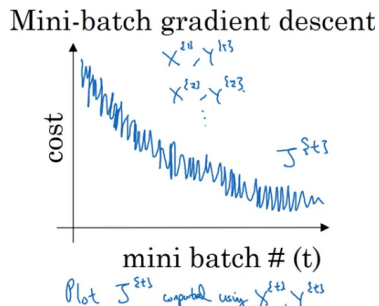
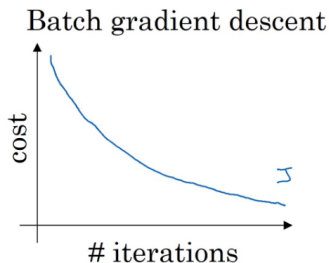
- No treinamento com todos os exemplos (*batch*), espera-se que o custo diminua em cada iteração
- No *mini-batch* a curva é muito mais ruidosa, mas ainda com tendência de queda
- Um *mini-batch* pode ser mais fácil, o seguinte mais difícil, etc., seguindo uma sequência aleatória

Entendendo Melhor o Gradiente Descendente em Mini-batch

- No treinamento com todos os exemplos (*batch*), espera-se que o custo diminua em cada iteração
- No *mini-batch* a curva é muito mais ruidosa, mas ainda com tendência de queda
- Um *mini-batch* pode ser mais fácil, o seguinte mais difícil, etc., seguindo uma sequência aleatória

Entendendo Melhor o Gradiente Descendente em Mini-batch

- No treinamento com todos os exemplos (*batch*), espera-se que o custo diminua em cada iteração
- No *mini-batch* a curva é muito mais ruidosa, mas ainda com tendência de queda
- Um *mini-batch* pode ser mais fácil, o seguinte mais difícil, etc., seguindo uma sequência aleatória



Entendendo Melhor o Gradiente Descendente em Mini-batch

- Tamanho do *mini-batch* é um hiperparâmetro a ser escolhido
- Extremos: tamanho de *mini-batch* m (*batch*) e 1 (gradiente descendente estocástico)
- O primeiro demora muito por iteração, o último faz um caminho muito irregular até chegar ao mínimo (podendo ficar dando voltas em torno do mínimo) e, principalmente, perde toda a vantagem da vetorização
- Melhor opção é um meio termo entre 1 e m : aprendizado não tão demorado, oscilação em região menor e mantém a vetorização

Entendendo Melhor o Gradiente Descendente em Mini-batch

- Tamanho do *mini-batch* é um hiperparâmetro a ser escolhido
- Extremos: tamanho de *mini-batch* m (*batch*) e 1 (gradiente descendente estocástico)
- O primeiro demora muito por iteração, o último faz um caminho muito irregular até chegar ao mínimo (podendo ficar dando voltas em torno do mínimo) e, principalmente, perde toda a vantagem da vetorização
- Melhor opção é um meio termo entre 1 e m : aprendizado não tão demorado, oscilação em região menor e mantém a vetorização

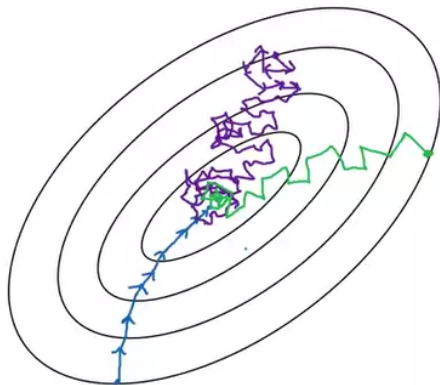
Entendendo Melhor o Gradiente Descendente em Mini-batch

- Tamanho do *mini-batch* é um hiperparâmetro a ser escolhido
- Extremos: tamanho de *mini-batch* m (*batch*) e 1 (gradiente descendente estocástico)
- O primeiro demora muito por iteração, o último faz um caminho muito irregular até chegar ao mínimo (podendo ficar dando voltas em torno do mínimo) e, principalmente, perde toda a vantagem da vetorização
- Melhor opção é um meio termo entre 1 e m : aprendizado não tão demorado, oscilação em região menor e mantém a vetorização

Entendendo Melhor o Gradiente Descendente em Mini-batch

- Tamanho do *mini-batch* é um hiperparâmetro a ser escolhido
- Extremos: tamanho de *mini-batch* m (*batch*) e 1 (gradiente descendente estocástico)
- O primeiro demora muito por iteração, o último faz um caminho muito irregular até chegar ao mínimo (podendo ficar dando voltas em torno do mínimo) e, principalmente, perde toda a vantagem da vetorização
- Melhor opção é um meio termo entre 1 e m : aprendizado não tão demorado, oscilação em região menor e mantém a vetorização

Entendendo Melhor o Gradiente Descendente em Mini-batch



Guia Geral

- Conjunto de treino pequeno ($m \leq 2000$), usar gradiente em *batch*
- Se não, usar *mini-batch*: tamanhos 64, 128, 256 e 512 são os mais comuns
- Garanta que o seu *mini-batch* vai caber na sua memória RAM ou de GPU

Guia Geral

- Conjunto de treino pequeno ($m \leq 2000$), usar gradiente em *batch*
- Se não, usar *mini-batch*: tamanhos 64, 128, 256 e 512 são os mais comuns
- Garanta que o seu *mini-batch* vai caber na sua memória RAM ou de GPU

Guia Geral

- Conjunto de treino pequeno ($m \leq 2000$), usar gradiente em *batch*
- Se não, usar *mini-batch*: tamanhos 64, 128, 256 e 512 são os mais comuns
- Garanta que o seu *mini-batch* vai caber na sua memória RAM ou de GPU

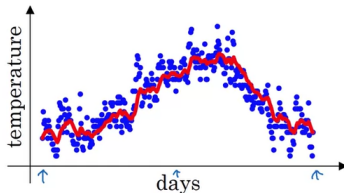
Outline

- 1 Gradiente Descendente por Mini-Batch
- 2 Média Ponderada Exponencialmente**
- 3 Métodos Avançados de Otimização
- 4 O Problema dos Ótimos Locais
- 5 Juntando as Peças

- [illegible]

- Veremos agora algoritmos de otimização que são mais rápidos do que o gradiente descendente
- Temperatura em Londres em determinado ano:

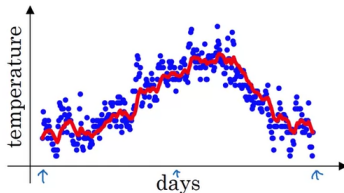
$\theta_1 = 40^\circ\text{F}$ $4^\circ\text{C} \leftarrow$
 $\theta_2 = 49^\circ\text{F}$ 9°C
 $\theta_3 = 45^\circ\text{F}$ $\vdots$
 $\vdots$
 $\theta_{180} = 60^\circ\text{F}$ 15°C
 $\theta_{181} = 56^\circ\text{F}$ $\vdots$
 $\vdots$



- Dado (azul) ruidoso
- Suavizado fazendo uma média (móvel) exponencialmente ponderada (vermelho):

- Veremos agora algoritmos de otimização que são mais rápidos do que o gradiente descendente
- Temperatura em Londres em determinado ano:

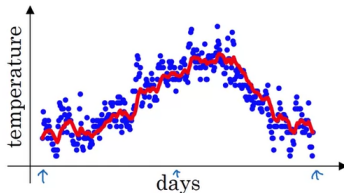
$\theta_1 = 40^\circ\text{F}$ $4^\circ\text{C} \leftarrow$
 $\theta_2 = 49^\circ\text{F}$ 9°C
 $\theta_3 = 45^\circ\text{F}$ $\vdots$
 $\vdots$
 $\theta_{180} = 60^\circ\text{F}$ 15°C
 $\theta_{181} = 56^\circ\text{F}$ $\vdots$
 $\vdots$



- Dado (azul) ruidoso
- Suavizado fazendo uma média (móvel) exponencialmente ponderada (vermelho):

- Veremos agora algoritmos de otimização que são mais rápidos do que o gradiente descendente
- Temperatura em Londres em determinado ano:

$\theta_1 = 40^\circ\text{F}$ $4^\circ\text{C} \leftarrow$
 $\theta_2 = 49^\circ\text{F}$ 9°C
 $\theta_3 = 45^\circ\text{F}$ $\vdots$
 $\vdots$
 $\theta_{180} = 60^\circ\text{F}$ 15°C
 $\theta_{181} = 56^\circ\text{F}$ $\vdots$
 $\vdots$



- Dado (azul) ruidoso
- Suavizado fazendo uma média (móvel) exponencialmente ponderada (vermelho):

$$V_0 = 0$$

$$V_1 = 0.9 * V_0 + 0.1 * \theta_1$$

$$V_2 = 0.9 * V_1 + 0.1 * \theta_2$$

...

$$V_t = 0.9 * V_{t-1} + 0.1 * \theta_t$$

- Fórmula geral:

$$V_t = \beta V_{t-1} + (1 - \beta)\theta_t$$

Exemplo anterior: $\beta = 0.9$

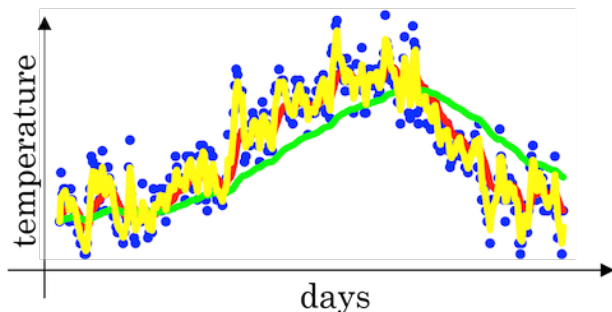
- $V_t \approx$ média sobre os $1/(1 - \beta)$ últimos dias:
 - $\beta = 0.9 \approx 10$ dias de temperatura (curva vermelha)
 - $\beta = 0.98 \approx 50$ dias ($1/(1-0.98)=50$) (curva verde)
 - $\beta = 0.5 \approx 2$ dias (curva amarela)

- Fórmula geral:

$$V_t = \beta V_{t-1} + (1 - \beta)\theta_t$$

Exemplo anterior: $\beta = 0.9$

- $V_t \approx$ média sobre os $1/(1 - \beta)$ últimos dias:
 - $\beta = 0.9 \approx 10$ dias de temperatura (curva vermelha)
 - $\beta = 0.98 \approx 50$ dias ($1/(1-0.98)=50$) (curva verde)
 - $\beta = 0.5 \approx 2$ dias (curva amarela)

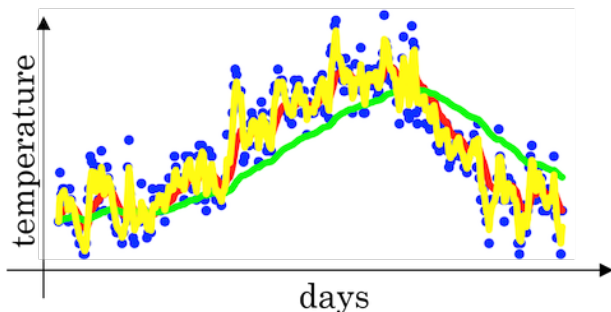


- Fórmula geral:

$$V_t = \beta V_{t-1} + (1 - \beta)\theta_t$$

Exemplo anterior: $\beta = 0.9$

- $V_t \approx$ média sobre os $1/(1 - \beta)$ últimos dias:
 - $\beta = 0.9 \approx 10$ dias de temperatura (curva vermelha)
 - $\beta = 0.98 \approx 50$ dias ($1/(1-0.98)=50$) (curva verde)
 - $\beta = 0.5 \approx 2$ dias (curva amarela)

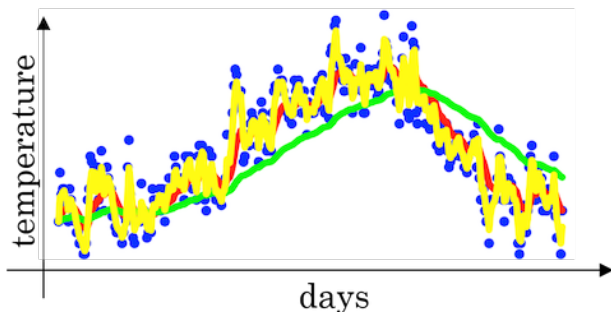


- Fórmula geral:

$$V_t = \beta V_{t-1} + (1 - \beta)\theta_t$$

Exemplo anterior: $\beta = 0.9$

- $V_t \approx$ média sobre os $1/(1 - \beta)$ últimos dias:
 - $\beta = 0.9 \approx 10$ dias de temperatura (curva vermelha)
 - $\beta = 0.98 \approx 50$ dias ($1/(1-0.98)=50$) (curva verde)
 - $\beta = 0.5 \approx 2$ dias (curva amarela)

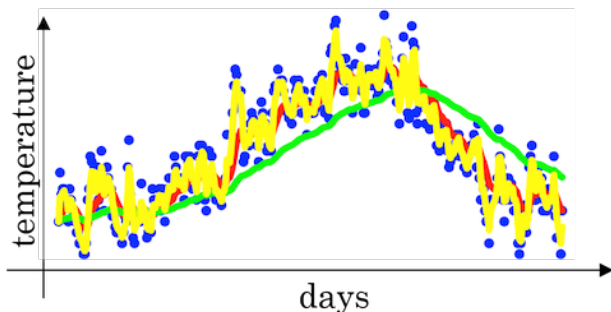


- Fórmula geral:

$$V_t = \beta V_{t-1} + (1 - \beta)\theta_t$$

Exemplo anterior: $\beta = 0.9$

- $V_t \approx$ média sobre os $1/(1 - \beta)$ últimos dias:
 - $\beta = 0.9 \approx 10$ dias de temperatura (curva vermelha)
 - $\beta = 0.98 \approx 50$ dias ($1/(1-0.98)=50$) (curva verde)
 - $\beta = 0.5 \approx 2$ dias (curva amarela)



- Quanto menor for β , mais ruidosa é a curva pois menos dias são ponderados
- Isso ocorre porque se dá um peso menor ao dia anterior (V_{t-1})
- Último cenário é mais suscetível a *outliers*, mas se adapta mais rapidamente a mudanças

- Quanto menor for β , mais ruidosa é a curva pois menos dias são ponderados
- Isso ocorre porque se dá um peso menor ao dia anterior (V_{t-1})
- Último cenário é mais suscetível a *outliers*, mas se adapta mais rapidamente a mudanças

- Quanto menor for β , mais ruidosa é a curva pois menos dias são ponderados
- Isso ocorre porque se dá um peso menor ao dia anterior (V_{t-1})
- Último cenário é mais suscetível a *outliers*, mas se adapta mais rapidamente a mudanças

Entendendo a Média Exponencialmente Ponderada

- Componente chave de vários algoritmos de otimização que usaremos
- Alguns dias de exemplo com $\beta = 0.9$:

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

$$v_{100} = 0.9v_{99} + 0.1\theta_{100}$$

$$v_{99} = 0.9v_{98} + 0.1\theta_{99}$$

$$v_{98} = 0.9v_{97} + 0.1\theta_{98}$$

...

Substituindo recursivamente:

$$\begin{aligned} v_{100} &= 0.1\theta_{100} + 0.9v_{99} \\ &= 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98}) \\ &= 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9(0.1\theta_{98} + 0.9v_{97})) \\ &= 0.1\theta_{100} + 0.1 \cdot 0.9 \cdot \theta_{99} + 0.1 \cdot (0.9)^2 \cdot \theta_{98} + 0.1 \cdot (0.9)^3 \cdot \theta_{97} + \\ &\quad + 0.1 \cdot (0.9)^4 \cdot \theta_{96} + \dots \end{aligned}$$

Entendendo a Média Exponencialmente Ponderada

- Componente chave de vários algoritmos de otimização que usaremos
- Alguns dias de exemplo com $\beta = 0.9$:

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

$$v_{100} = 0.9v_{99} + 0.1\theta_{100}$$

$$v_{99} = 0.9v_{98} + 0.1\theta_{99}$$

$$v_{98} = 0.9v_{97} + 0.1\theta_{98}$$

...

Substituindo recursivamente:

$$\begin{aligned} v_{100} &= 0.1\theta_{100} + 0.9v_{99} \\ &= 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98}) \\ &= 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9(0.1\theta_{98} + 0.9v_{97})) \\ &= 0.1\theta_{100} + 0.1 \cdot 0.9 \cdot \theta_{99} + 0.1 \cdot (0.9)^2 \cdot \theta_{98} + 0.1 \cdot (0.9)^3 \cdot \theta_{97} + \\ &\quad + 0.1 \cdot (0.9)^4 \cdot \theta_{96} + \dots \end{aligned}$$

Entendendo a Média Exponencialmente Ponderada

- Componente chave de vários algoritmos de otimização que usaremos
- Alguns dias de exemplo com $\beta = 0.9$:

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

$$v_{100} = 0.9v_{99} + 0.1\theta_{100}$$

$$v_{99} = 0.9v_{98} + 0.1\theta_{99}$$

$$v_{98} = 0.9v_{97} + 0.1\theta_{98}$$

...

Substituindo recursivamente:

$$\begin{aligned} v_{100} &= 0.1\theta_{100} + 0.9v_{99} \\ &= 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98}) \\ &= 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9(0.1\theta_{98} + 0.9v_{97})) \\ &= 0.1\theta_{100} + 0.1 \cdot 0.9 \cdot \theta_{99} + 0.1 \cdot (0.9)^2 \cdot \theta_{98} + 0.1 \cdot (0.9)^3 \cdot \theta_{97} + \\ &\quad + 0.1 \cdot (0.9)^4 \cdot \theta_{96} + \dots \end{aligned}$$

Entendendo a Média Exponencialmente Ponderada

- Componente chave de vários algoritmos de otimização que usaremos
- Alguns dias de exemplo com $\beta = 0.9$:

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t$$

$$v_{100} = 0.9v_{99} + 0.1\theta_{100}$$

$$v_{99} = 0.9v_{98} + 0.1\theta_{99}$$

$$v_{98} = 0.9v_{97} + 0.1\theta_{98}$$

...

Substituindo recursivamente:

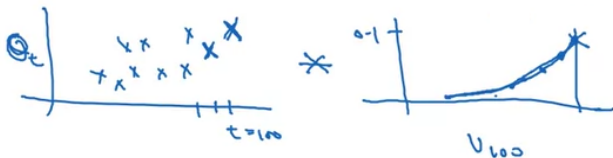
$$\begin{aligned} v_{100} &= 0.1\theta_{100} + 0.9v_{99} \\ &= 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9v_{98}) \\ &= 0.1\theta_{100} + 0.9(0.1\theta_{99} + 0.9(0.1\theta_{98} + 0.9v_{97})) \\ &= 0.1\theta_{100} + 0.1 \cdot 0.9 \cdot \theta_{99} + 0.1 \cdot (0.9)^2 \cdot \theta_{98} + 0.1 \cdot (0.9)^3 \cdot \theta_{97} + \\ &\quad + 0.1 \cdot (0.9)^4 \cdot \theta_{96} + \dots \end{aligned}$$

Entendendo a Média Exponencialmente Ponderada

- Note que é uma média ponderada com pesos decaindo exponencialmente (origem do nome)
- Multiplicação ponto-a-ponto entre o dado original e um decaimento exponencial:

Entendendo a Média Exponencialmente Ponderada

- Note que é uma média ponderada com pesos decaindo exponencialmente (origem do nome)
- Multiplicação ponto-a-ponto entre o dado original e um decaimento exponencial:



Entendendo a Média Exponencialmente Ponderada

- Quantos dias essa média “cobre”?
- $0.9^{10} \approx 0.35 \approx 1/e$: após 10 dias o peso cai aproximadamente um terço ($1/e$) - isso é chamado de **e-folding**
- Em geral, $(1 - \epsilon)^{1/\epsilon} = 1/e$ (no caso $\epsilon = 0.1$)
- Por exemplo, $0.98^{50} \approx 0.35 \approx 1/e$: peso cai para um terço após 50 dias
- Daqui que tiramos a fórmula do slide anterior
- NOTA: Soma dos pesos bem próxima de 1, exceto por um fator chamado que **correção de viés**

Entendendo a Média Exponencialmente Ponderada

- Quantos dias essa média “cobre”?
- $0.9^{10} \approx 0.35 \approx 1/e$: após 10 dias o peso cai aproximadamente um terço ($1/e$) - isso é chamado de **e-folding**
- Em geral, $(1 - \epsilon)^{1/\epsilon} = 1/e$ (no caso $\epsilon = 0.1$)
- Por exemplo, $0.98^{50} \approx 0.35 \approx 1/e$: peso cai para um terço após 50 dias
- Daqui que tiramos a fórmula do slide anterior
- NOTA: Soma dos pesos bem próxima de 1, exceto por um fator chamado que **correção de viés**

Entendendo a Média Exponencialmente Ponderada

- Quantos dias essa média “cobre”?
- $0.9^{10} \approx 0.35 \approx 1/e$: após 10 dias o peso cai aproximadamente um terço ($1/e$) - isso é chamado de **e-folding**
- Em geral, $(1 - \epsilon)^{1/\epsilon} = 1/e$ (no caso $\epsilon = 0.1$)
- Por exemplo, $0.98^{50} \approx 0.35 \approx 1/e$: peso cai para um terço após 50 dias
- Daqui que tiramos a fórmula do slide anterior
- NOTA: Soma dos pesos bem próxima de 1, exceto por um fator chamado que **correção de viés**

Entendendo a Média Exponencialmente Ponderada

- Quantos dias essa média “cobre”?
- $0.9^{10} \approx 0.35 \approx 1/e$: após 10 dias o peso cai aproximadamente um terço ($1/e$) - isso é chamado de **e-folding**
- Em geral, $(1 - \epsilon)^{1/\epsilon} = 1/e$ (no caso $\epsilon = 0.1$)
- Por exemplo, $0.98^{50} \approx 0.35 \approx 1/e$: peso cai para um terço após 50 dias
- Daqui que tiramos a fórmula do slide anterior
- NOTA: Soma dos pesos bem próxima de 1, exceto por um fator chamado que **correção de viés**

Entendendo a Média Exponencialmente Ponderada

- Quantos dias essa média “cobre”?
- $0.9^{10} \approx 0.35 \approx 1/e$: após 10 dias o peso cai aproximadamente um terço ($1/e$) - isso é chamado de **e-folding**
- Em geral, $(1 - \epsilon)^{1/\epsilon} = 1/e$ (no caso $\epsilon = 0.1$)
- Por exemplo, $0.98^{50} \approx 0.35 \approx 1/e$: peso cai para um terço após 50 dias
- Daqui que tiramos a fórmula do slide anterior
- NOTA: Soma dos pesos bem próxima de 1, exceto por um fator chamado que **correção de viés**

Entendendo a Média Exponencialmente Ponderada

- Quantos dias essa média “cobre”?
- $0.9^{10} \approx 0.35 \approx 1/e$: após 10 dias o peso cai aproximadamente um terço ($1/e$) - isso é chamado de **e-folding**
- Em geral, $(1 - \epsilon)^{1/\epsilon} = 1/e$ (no caso $\epsilon = 0.1$)
- Por exemplo, $0.98^{50} \approx 0.35 \approx 1/e$: peso cai para um terço após 50 dias
- Daqui que tiramos a fórmula do slide anterior
- NOTA: Soma dos pesos bem próxima de 1, exceto por um fator chamado que **correção de viés**

Implementação

$$v_\theta = 0$$

Repita{

 Obtenha θ_t

$$v_\theta := \beta v_\theta + (1 - \beta)\theta_t$$

}

- Não tão preciso quanto calcular a média convencionalmente, mas altamente econômico em uso de memória

Implementação

$$v_\theta = 0$$

Repita{

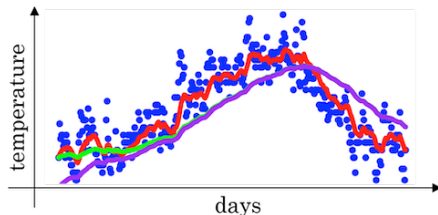
Obtenha θ_t

$$v_\theta := \beta v_\theta + (1 - \beta)\theta_t$$

}

- Não tão preciso quanto calcular a média convencionalmente, mas altamente econômico em uso de memória

Correção de Viés

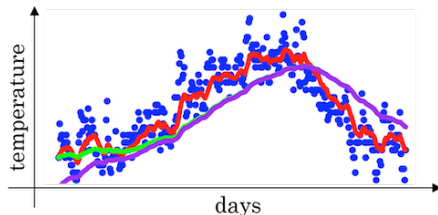


- Para $\beta = 0.98$, temos na verdade a curva púrpura, que no início não acompanha corretamente o esperado (verde)
- Como definimos $v_0 = 0$:

$$V_1 = 0.98 * V_0 + 0.02 * \theta_1 = 0.02 * \theta_1$$

$$V_2 = 0.98 * V_1 + 0.02 * \theta_2 = 0.0196 * \theta_1 + 0.02 * \theta_2$$
- V_2 tem um valor muito menor do que θ_1 e θ_2 (não condizente com uma média)

Correção de Viés

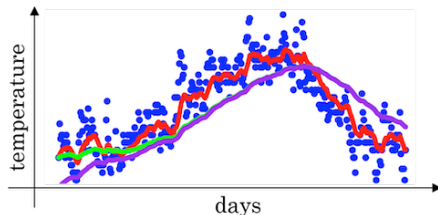


- Para $\beta = 0.98$, temos na verdade a curva púrpura, que no início não acompanha corretamente o esperado (verde)
- Como definimos $v_0 = 0$:

$$V_1 = 0.98 * V_0 + 0.02 * \theta_1 = 0.02 * \theta_1$$

$$V_2 = 0.98 * V_1 + 0.02 * \theta_2 = 0.0196 * \theta_1 + 0.02 * \theta_2$$
- V_2 tem um valor muito menor do que θ_1 e θ_2 (não condizente com uma média)

Correção de Viés



- Para $\beta = 0.98$, temos na verdade a curva púrpura, que no início não acompanha corretamente o esperado (verde)
- Como definimos $v_0 = 0$:

$$V_1 = 0.98 * V_0 + 0.02 * \theta_1 = 0.02 * \theta_1$$

$$V_2 = 0.98 * V_1 + 0.02 * \theta_2 = 0.0196 * \theta_1 + 0.02 * \theta_2$$
- V_2 tem um valor muito menor do que θ_1 e θ_2 (não condizente com uma média)

Correção de Viés

- Em vez de V_t , tomar $\frac{V_t}{1-\beta^t}$

- No exemplo, para $t = 2$, temos

$$1 - \beta^t = 1 - 0.98^2 = 0.0396$$

V_2 após a correção será então

$$\frac{V_2}{0.0396} = \frac{0.0196 * \theta_1 + 0.02 * \theta_2}{0.0396}$$

- Note que quando t se torna grande, $\beta^t \rightarrow 0$ e essa correção já não faz mais diferença
- Nos primeiros dias, porém, isso se torna algo relevante

Correção de Viés

- Em vez de V_t , tomar $\frac{V_t}{1-\beta^t}$
- No exemplo, para $t = 2$, temos

$$1 - \beta^t = 1 - 0.98^2 = 0.0396$$

V_2 após a correção será então

$$\frac{V_2}{0.0396} = \frac{0.0196 * \theta_1 + 0.02 * \theta_2}{0.0396}$$

- Note que quando t se torna grande, $\beta^t \rightarrow 0$ e essa correção já não faz mais diferença
- Nos primeiros dias, porém, isso se torna algo relevante

Correção de Viés

- Em vez de V_t , tomar $\frac{V_t}{1-\beta^t}$
- No exemplo, para $t = 2$, temos

$$1 - \beta^t = 1 - 0.98^2 = 0.0396$$

V_2 após a correção será então

$$\frac{V_2}{0.0396} = \frac{0.0196 * \theta_1 + 0.02 * \theta_2}{0.0396}$$

- Note que quando t se torna grande, $\beta^t \rightarrow 0$ e essa correção já não faz mais diferença
- Nos primeiros dias, porém, isso se torna algo relevante

Correção de Viés

- Em vez de V_t , tomar $\frac{V_t}{1-\beta^t}$
- No exemplo, para $t = 2$, temos

$$1 - \beta^t = 1 - 0.98^2 = 0.0396$$

V_2 após a correção será então

$$\frac{V_2}{0.0396} = \frac{0.0196 * \theta_1 + 0.02 * \theta_2}{0.0396}$$

- Note que quando t se torna grande, $\beta^t \rightarrow 0$ e essa correção já não faz mais diferença
- Nos primeiros dias, porém, isso se torna algo relevante

Outline

- 1 Gradiente Descendente por Mini-Batch
- 2 Média Ponderada Exponencialmente
- 3 Métodos Avançados de Otimização**
- 4 O Problema dos Ótimos Locais
- 5 Juntando as Peças

Gradiente Descendente com Momento

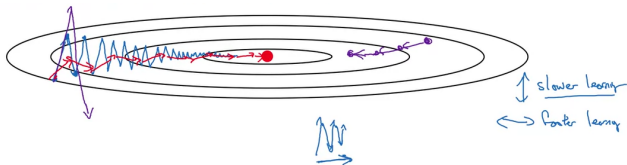
- Geralmente funciona melhor que o gradiente descendente padrão
- Média exponencialmente ponderada do gradiente para fazer atualizações
- Comum que a trajetória do gradiente descendente clássico tenha uma forte oscilação:
- Se usar uma taxa de aprendizado maior, o algoritmo pode inclusive divergir

Gradiente Descendente com Momento

- Geralmente funciona melhor que o gradiente descendente padrão
- Média exponencialmente ponderada do gradiente para fazer atualizações
- Comum que a trajetória do gradiente descendente clássico tenha uma forte oscilação:
- Se usar uma taxa de aprendizado maior, o algoritmo pode inclusive divergir

Gradiente Descendente com Momento

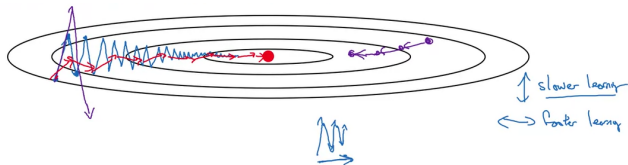
- Geralmente funciona melhor que o gradiente descendente padrão
- Média exponencialmente ponderada do gradiente para fazer atualizações
- Comum que a trajetória do gradiente descendente clássico tenha uma forte oscilação:



- Se usar uma taxa de aprendizado maior, o algoritmo pode inclusive divergir

Gradiente Descendente com Momento

- Geralmente funciona melhor que o gradiente descendente padrão
- Média exponencialmente ponderada do gradiente para fazer atualizações
- Comum que a trajetória do gradiente descendente clássico tenha uma forte oscilação:



- Se usar uma taxa de aprendizado maior, o algoritmo pode inclusive divergir

Gradiente Descendente com Momento

- Média dos gradientes suaviza as oscilações, ao mesmo tempo que a derivada continua apontando na direção do mínimo (aprendizado acelerado)

$$v_{dw} = 0, v_{db} = 0$$

Na iteração t :

Calcule dW , db no *mini-batch* corrente

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

- Analogia física: momento gera fricção que previne um movimento desordenado de uma bola descendo uma superfície

Gradiente Descendente com Momento

- Média dos gradientes suaviza as oscilações, ao mesmo tempo que a derivada continua apontando na direção do mínimo (aprendizado acelerado)

$$v_{dw} = 0, v_{db} = 0$$

Na iteração t :

Calcule dW , db no *mini-batch* corrente

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

- Analogia física: momento gera fricção que previne um movimento desordenado de uma bola descendo uma superfície

Gradiente Descendente com Momento

- Média dos gradientes suaviza as oscilações, ao mesmo tempo que a derivada continua apontando na direção do mínimo (aprendizado acelerado)

$$v_{dw} = 0, v_{db} = 0$$

Na iteração t :

Calcule dW , db no *mini-batch* corrente

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

- Analogia física: momento gera fricção que previne um movimento desordenado de uma bola descendo uma superfície

Gradiente Descendente com Momento

- Média dos gradientes suaviza as oscilações, ao mesmo tempo que a derivada continua apontando na direção do mínimo (aprendizado acelerado)

$$v_{dw} = 0, v_{db} = 0$$

Na iteração t :

Calcule dW , db no *mini-batch* corrente

$$v_{dW} = \beta v_{dW} + (1 - \beta) dW$$

$$v_{db} = \beta v_{db} + (1 - \beta) db$$

$$W = W - \alpha v_{dW}, \quad b = b - \alpha v_{db}$$

- Analogia física: momento gera fricção que previne um movimento desordenado de uma bola descendo uma superfície

Hiperparâmetros

- Hiperparâmetro a mais β , comum usar $\beta = 0.9$, mas pode ser testado
- Não se costuma usar o fator de correção de viés: como ele só vai interferir nas primeiras iterações, acaba sendo irrelevante
- É comum também na literatura que se omita o termo $(1 - \beta)$:

$$v_{dW} = \beta v_{dW} + dW$$

- Implica em um fator $1/(1 - \beta)$ aplicado sobre a taxa de aprendizado α (vai ter um valor ótimo diferente portanto)
- A versão com $(1 - \beta)$ é mais intuitiva

Hiperparâmetros

- Hiperparâmetro β , comum usar $\beta = 0.9$, mas pode ser testado
- Não se costuma usar o fator de correção de viés: como ele só vai interferir nas primeiras iterações, acaba sendo irrelevante
- É comum também na literatura que se omita o termo $(1 - \beta)$:

$$v_{dW} = \beta v_{dW} + dW$$

- Implica em um fator $1/(1 - \beta)$ aplicado sobre a taxa de aprendizado α (vai ter um valor ótimo diferente portanto)
- A versão com $(1 - \beta)$ é mais intuitiva

Hiperparâmetros

- Hiperparâmetro a mais β , comum usar $\beta = 0.9$, mas pode ser testado
- Não se costuma usar o fator de correção de viés: como ele só vai interferir nas primeiras iterações, acaba sendo irrelevante
- É comum também na literatura que se omita o termo $(1 - \beta)$:

$$v_{dW} = \beta v_{dW} + dW$$

- Implica em um fator $1/(1 - \beta)$ aplicado sobre a taxa de aprendizado α (vai ter um valor ótimo diferente portanto)
- A versão com $(1 - \beta)$ é mais intuitiva

Hiperparâmetros

- Hiperparâmetro β , comum usar $\beta = 0.9$, mas pode ser testado
- Não se costuma usar o fator de correção de viés: como ele só vai interferir nas primeiras iterações, acaba sendo irrelevante
- É comum também na literatura que se omita o termo $(1 - \beta)$:

$$v_{dW} = \beta v_{dW} + dW$$

- Implica em um fator $1/(1 - \beta)$ aplicado sobre a taxa de aprendizado α (vai ter um valor ótimo diferente portanto)
- A versão com $(1 - \beta)$ é mais intuitiva

Hiperparâmetros

- Hiperparâmetro β , comum usar $\beta = 0.9$, mas pode ser testado
- Não se costuma usar o fator de correção de viés: como ele só vai interferir nas primeiras iterações, acaba sendo irrelevante
- É comum também na literatura que se omita o termo $(1 - \beta)$:

$$v_{dW} = \beta v_{dW} + dW$$

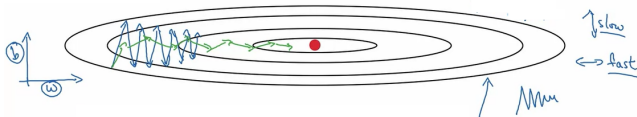
- Implica em um fator $1/(1 - \beta)$ aplicado sobre a taxa de aprendizado α (vai ter um valor ótimo diferente portanto)
- A versão com $(1 - \beta)$ é mais intuitiva

RMSprop (Root Mean Square)

- Visa amortecer oscilação na vertical (azul)
- Suponha apenas dois parâmetros, w na horizontal e b na vertical:
- Desacelera na vertical e acelera (ou pelo menos evita desacelerar) na horizontal (verde)
- Média ponderada exponencialmente porém sobre o quadrado (ponto a ponto) do gradiente (e depois tira a raiz)

RMSprop (Root Mean Square)

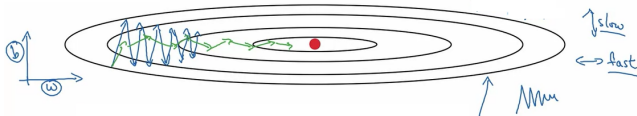
- Visa amortecer oscilação na vertical (azul)
- Suponha apenas dois parâmetros, w na horizontal e b na vertical:



- Desacelera na vertical e acelera (ou pelo menos evita desacelerar) na horizontal (verde)
- Média ponderada exponencialmente porém sobre o quadrado (ponto a ponto) do gradiente (e depois tira a raiz)

RMSprop (Root Mean Square)

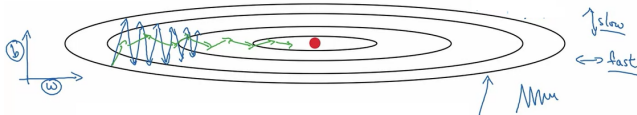
- Visa amortecer oscilação na vertical (azul)
- Suponha apenas dois parâmetros, w na horizontal e b na vertical:



- Desacelera na vertical e acelera (ou pelo menos evita desacelerar) na horizontal (verde)
- Média ponderada exponencialmente porém sobre o quadrado (ponto a ponto) do gradiente (e depois tira a raiz)

RMSprop (Root Mean Square)

- Visa amortecer oscilação na vertical (azul)
- Suponha apenas dois parâmetros, w na horizontal e b na vertical:



- Desacelera na vertical e acelera (ou pelo menos evita desacelerar) na horizontal (verde)
- Média ponderada exponencialmente porém sobre o quadrado (ponto a ponto) do gradiente (e depois tira a raiz)

RMSprop

Em cada iteração t :

Calcule dW , db no *mini-batch*

$$S_{dW} = (\beta * S_{dW}) + (1 - \beta) * dW^2$$

$$S_{db} = (\beta * S_{db}) + (1 - \beta) * db^2$$

$$W = W - \alpha * dW / \sqrt{S_{dW} + \epsilon}$$

$$b = b - \alpha * db / \sqrt{S_{db} + \epsilon}$$

NOTA: ϵ é um número pequeno (10^{-8}) que evita divisão por zero

- Divide dw (horizontal) por um número menor, já que a derivada (variação) nessa direção é menor
- Oposto ocorre com db já que a derivada na vertical é maior, devido à oscilação

RMSprop

Em cada iteração t :

Calcule dW , db no *mini-batch*

$$S_{dW} = (\beta * S_{dW}) + (1 - \beta) * dW^2$$

$$S_{db} = (\beta * S_{db}) + (1 - \beta) * db^2$$

$$W = W - \alpha * dW / \sqrt{S_{dW} + \epsilon}$$

$$b = b - \alpha * db / \sqrt{S_{db} + \epsilon}$$

NOTA: ϵ é um número pequeno (10^{-8}) que evita divisão por zero

- Divide dw (horizontal) por um número menor, já que a derivada (variação) nessa direção é menor
- Oposto ocorre com db já que a derivada na vertical é maior, devido à oscilação

RMSprop

Em cada iteração t :

Calcule dW , db no *mini-batch*

$$S_{dW} = (\beta * S_{dW}) + (1 - \beta) * dW^2$$

$$S_{db} = (\beta * S_{db}) + (1 - \beta) * db^2$$

$$W = W - \alpha * dW / \sqrt{S_{dW} + \epsilon}$$

$$b = b - \alpha * db / \sqrt{S_{db} + \epsilon}$$

NOTA: ϵ é um número pequeno (10^{-8}) que evita divisão por zero

- Divide dw (horizontal) por um número menor, já que a derivada (variação) nessa direção é menor
- Oposto ocorre com db já que a derivada na vertical é maior, devido à oscilação

RMSprop

- Permite que se use uma taxa de aprendizado maior, sem o risco de divergência, acelerando o algoritmo em geral
- NOTA: Em geral, não temos apenas w e b , mas sim um conjunto grande de parâmetros responsável por uma tendência de oscilação mais vertical e um outro se aproximando mais do mínimo horizontalmente
- CURIOSIDADE: O método não surgiu a primeira vez em um artigo científico, mas sim em um curso no Coursera dado por Hinton

RMSprop

- Permite que se use uma taxa de aprendizado maior, sem o risco de divergência, acelerando o algoritmo em geral
- NOTA: Em geral, não temos apenas w e b , mas sim um conjunto grande de parâmetros responsável por uma tendência de oscilação mais vertical e um outro se aproximando mais do mínimo horizontalmente
- CURIOSIDADE: O método não surgiu a primeira vez em um artigo científico, mas sim em um curso no Coursera dado por Hinton

RMSprop

- Permite que se use uma taxa de aprendizado maior, sem o risco de divergência, acelerando o algoritmo em geral
- NOTA: Em geral, não temos apenas w e b , mas sim um conjunto grande de parâmetros responsável por uma tendência de oscilação mais vertical e um outro se aproximando mais do mínimo horizontalmente
- CURIOSIDADE: O método não surgiu a primeira vez em um artigo científico, mas sim em um curso no Coursera dado por Hinton

Adam (Adaptive Moment Estimation)

- RMSprop + momento

$$V_{dW} = 0, V_{db} = 0, S_{dW} = 0, S_{db} = 0$$

Em cada iteração t :

Calcule dW , db usando o *mini-batch*

Momento:

$$V_{dW} = (\beta_1 * V_{dW}) + (1 - \beta_1) * dW, \quad V_{db} = (\beta_1 * V_{db}) + (1 - \beta_1) * db$$

RMSprop:

$$S_{dW} = (\beta_2 * S_{dW}) + (1 - \beta_2) * dW^2, \quad S_{db} = (\beta_2 * S_{db}) + (1 - \beta_2) * db^2$$

Correção de viés:

$$V_{dW_c} = V_{dW} / (1 - \beta_1^t), \quad V_{db_c} = V_{db} / (1 - \beta_1^t)$$

$$S_{dW_c} = S_{dW} / (1 - \beta_2^t), \quad S_{db_c} = S_{db} / (1 - \beta_2^t)$$

$$W = W - \alpha * V_{dW_c} / \sqrt{S_{dW_c} + \epsilon}, \quad b = b - \alpha * V_{db_c} / \sqrt{S_{db_c} + \epsilon}$$

Hiperparâmetros

Hyperparâmetro	Valor sugerido
α	ajustar
β_1 (dW)	0.9
β_2 (dW^2)	0.999
ϵ	10^{-8}

Decaimento da Taxa de Aprendizado

- Suponha um *mini-batch* com tamanho p.ex. 64 ou 128
- Vai em direção ao mínimo, mas oscilando
- Com um tamanho de passo fixo, pode ficar dando voltas em torno do mínimo sem convergir
- Ir reduzindo a taxa de aprendizado restringe espaço em torno do mínimo

Decaimento da Taxa de Aprendizado

- Suponha um *mini-batch* com tamanho p.ex. 64 ou 128
- Vai em direção ao mínimo, mas oscilando
- Com um tamanho de passo fixo, pode ficar dando voltas em torno do mínimo sem convergir
- Ir reduzindo a taxa de aprendizado restringe espaço em torno do mínimo

Decaimento da Taxa de Aprendizado

- Suponha um *mini-batch* com tamanho p.ex. 64 ou 128
- Vai em direção ao mínimo, mas oscilando
- Com um tamanho de passo fixo, pode ficar dando voltas em torno do mínimo sem convergir
- Ir reduzindo a taxa de aprendizado restringe espaço em torno do mínimo

Decaimento da Taxa de Aprendizado

- Suponha um *mini-batch* com tamanho p.ex. 64 ou 128
- Vai em direção ao mínimo, mas oscilando
- Com um tamanho de passo fixo, pode ficar dando voltas em torno do mínimo sem convergir
- Ir reduzindo a taxa de aprendizado restringe espaço em torno do mínimo

Decaimento da Taxa de Aprendizizado

- Uma **época** é a passagem por todo o conjunto de treino (todos os *batches*):



Fórmula:

$$\alpha = \frac{1}{1 + \text{taxa_decaimento} * \text{num_epoca}} \alpha_0$$

Exemplo com $\alpha_0 = 0.2$ e $\text{taxa_decaimento} = 1$:

Época	α
1	0.1
2	0.067
3	0.05
4	0.04
$\vdots$	$\vdots$

Decaimento da Taxa de Aprendizado

- Uma **época** é a passagem por todo o conjunto de treino (todos os *batches*):



Fórmula:

$$\alpha = \frac{1}{1 + \text{taxa_decaimento} * \text{num_epoca}} \alpha_0$$

Exemplo com $\alpha_0 = 0.2$ e $\text{taxa_decaimento} = 1$:

Época	α
1	0.1
2	0.067
3	0.05
4	0.04
$\vdots$	$\vdots$

Decaimento da Taxa de Aprendizado

- Uma **época** é a passagem por todo o conjunto de treino (todos os *batches*):



Fórmula:

$$\alpha = \frac{1}{1 + \text{taxa_decaimento} * \text{num_epoca}} \alpha_0$$

Exemplo com $\alpha_0 = 0.2$ e $\text{taxa_decaimento} = 1$:

Época	α
1	0.1
2	0.067
3	0.05
4	0.04
$\vdots$	$\vdots$

Outras Fórmulas

Fórmula	Descrição
$\alpha = 0.95^{\text{num_epoca}} \alpha_0$	Dcaimento exponencial
$\alpha = \frac{k}{\sqrt{\text{num_epoca}}} \alpha_0$ ou $\frac{k}{\sqrt{t}} \alpha_0$	Dcaimento polinomial
Dcaimento discreto	Constante por parte
Manual	-

NOTA 1: t se refere ao número do *mini-batch*

NOTA 2: Decaimento manual só é viável para um único modelo treinado (no máximo alguns poucos)

NOTA 3: α_0 é mais um hiperparâmetro a ser ajustado

NOTA 4: Na prática, embora o decaimento ajude um pouco, a escolha adequada de α é mais importante

Outras Fórmulas

Fórmula	Descrição
$\alpha = 0.95^{\text{num_epoca}} \alpha_0$	Dcaimento exponencial
$\alpha = \frac{k}{\sqrt{\text{num_epoca}}} \alpha_0$ ou $\frac{k}{\sqrt{t}} \alpha_0$	Dcaimento polinomial
Dcaimento discreto	Constante por parte
Manual	-

NOTA 1: t se refere ao número do *mini-batch*

NOTA 2: Dcaimento manual só é viável para um único modelo treinado (no máximo alguns poucos)

NOTA 3: α_0 é mais um hiperparâmetro a ser ajustado

NOTA 4: Na prática, embora o dcaimento ajude um pouco, a escolha adequada de α é mais importante

Outras Fórmulas

Fórmula	Descrição
$\alpha = 0.95^{\text{num_epoca}} \alpha_0$	Dcaimento exponencial
$\alpha = \frac{k}{\sqrt{\text{num_epoca}}} \alpha_0$ ou $\frac{k}{\sqrt{t}} \alpha_0$	Dcaimento polinomial
Dcaimento discreto	Constante por parte
Manual	-

NOTA 1: t se refere ao número do *mini-batch*

NOTA 2: Dcaimento manual só é viável para um único modelo treinado (no máximo alguns poucos)

NOTA 3: α_0 é mais um hiperparâmetro a ser ajustado

NOTA 4: Na prática, embora o dcaimento ajude um pouco, a escolha adequada de α é mais importante

Outras Fórmulas

Fórmula	Descrição
$\alpha = 0.95^{\text{num_epoca}} \alpha_0$	Decaimento exponencial
$\alpha = \frac{k}{\sqrt{\text{num_epoca}}} \alpha_0$ ou $\frac{k}{\sqrt{t}} \alpha_0$	Decaimento polinomial
Decaimento discreto	Constante por parte
Manual	-

NOTA 1: t se refere ao número do *mini-batch*

NOTA 2: Decaimento manual só é viável para um único modelo treinado (no máximo alguns poucos)

NOTA 3: α_0 é mais um hiperparâmetro a ser ajustado

NOTA 4: Na prática, embora o decaimento ajude um pouco, a escolha adequada de α é mais importante

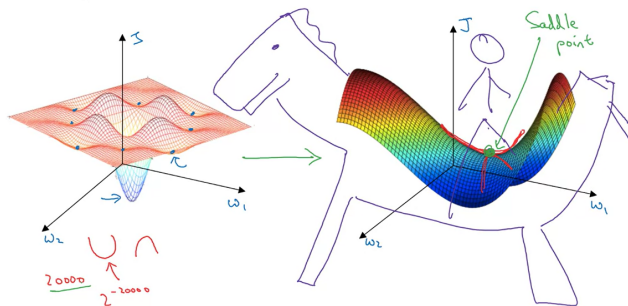
Outline

- 1 Gradiente Descendente por Mini-Batch
- 2 Média Ponderada Exponencialmente
- 3 Métodos Avançados de Otimização
- 4 O Problema dos Ótimos Locais**
- 5 Juntando as Peças

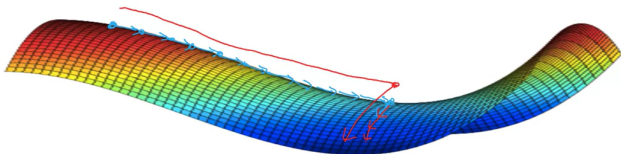
- Ótimos locais eram uma preocupação grande nos primórdios do *deep learning*; isso foi mudando!
- É fácil se observarem ótimos locais em baixa dimensão
- Mas em redes neurais profundas, a maioria dos pontos onde o gradiente se anula são pontos de sela e não de ótimo

- Ótimos locais eram uma preocupação grande nos primórdios do *deep learning*; isso foi mudando!
- É fácil se observarem ótimos locais em baixa dimensão
- Mas em redes neurais profundas, a maioria dos pontos onde o gradiente se anula são pontos de sela e não de ótimo

- Ótimos locais eram uma preocupação grande nos primórdios do *deep learning*; isso foi mudando!
- É fácil se observarem ótimos locais em baixa dimensão
- Mas em redes neurais profundas, a maioria dos pontos onde o gradiente se anula são pontos de sela e não de ótimo

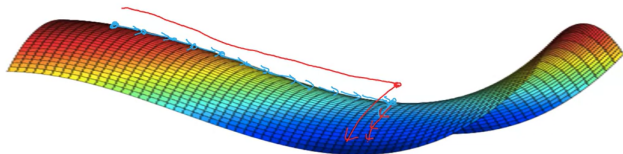


- O que é de fato um problema são *plateaus*, regiões em que a derivada é próxima de zero por muito tempo



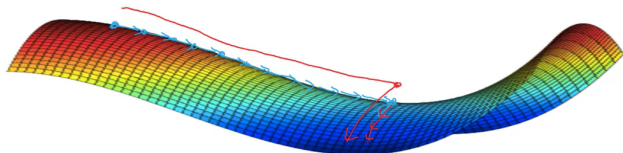
- Em resumo:
 - Ficar preso em ótimos locais ruins é bem improvável em redes neurais profundas
 - *Plateaus* sim são um problema pois o caminho do gradiente se torna extremamente lento; métodos como momento, RMSprop e Adam são muito importantes aqui
 - Tudo que apresentamos aqui são intuições; não temos nenhum conhecimento profundo sobre esses espaços de dimensões tão altas

- O que é de fato um problema são *plateaus*, regiões em que a derivada é próxima de zero por muito tempo



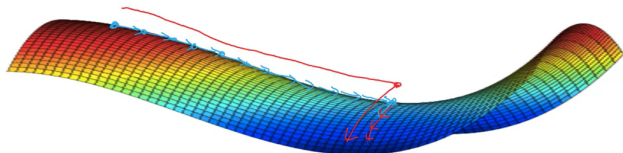
- Em resumo:
 - Ficar preso em ótimos locais ruins é bem improvável em redes neurais profundas
 - *Plateaus* sim são um problema pois o caminho do gradiente se torna extremamente lento; métodos como momento, RMSprop e Adam são muito importantes aqui
 - Tudo que apresentamos aqui são intuições; não temos nenhum conhecimento profundo sobre esses espaços de dimensões tão altas

- O que é de fato um problema são *plateaus*, regiões em que a derivada é próxima de zero por muito tempo



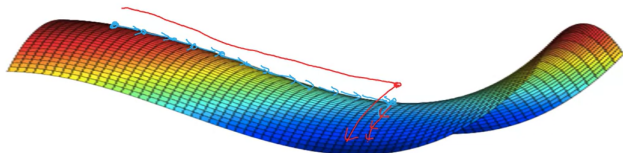
- Em resumo:
 - Ficar preso em ótimos locais ruins é bem improvável em redes neurais profundas
 - *Plateaus* sim são um problema pois o caminho do gradiente se torna extremamente lento; métodos como momento, RMSprop e Adam são muito importantes aqui
 - Tudo que apresentamos aqui são intuições; não temos nenhum conhecimento profundo sobre esses espaços de dimensões tão altas

- O que é de fato um problema são *plateaus*, regiões em que a derivada é próxima de zero por muito tempo



- Em resumo:
 - Ficar preso em ótimos locais ruins é bem improvável em redes neurais profundas
 - *Plateaus* sim são um problema pois o caminho do gradiente se torna extremamente lento; métodos como momento, RMSprop e Adam são muito importantes aqui
 - Tudo que apresentamos aqui são intuições; não temos nenhum conhecimento profundo sobre esses espaços de dimensões tão altas

- O que é de fato um problema são *plateaus*, regiões em que a derivada é próxima de zero por muito tempo



- Em resumo:
 - Ficar preso em ótimos locais ruins é bem improvável em redes neurais profundas
 - *Plateaus* sim são um problema pois o caminho do gradiente se torna extremamente lento; métodos como momento, RMSprop e Adam são muito importantes aqui
 - Tudo que apresentamos aqui são intuições; não temos nenhum conhecimento profundo sobre esses espaços de dimensões tão altas

Outline

- 1 Gradiente Descendente por Mini-Batch
- 2 Média Ponderada Exponencialmente
- 3 Métodos Avançados de Otimização
- 4 O Problema dos Ótimos Locais
- 5 Juntando as Peças

Já vimos os passos gerais de uma rede neural:

- ❶ Inicializar parâmetros / Definir hiperparâmetros
- ❷ Itere por `num_iter`:
 - ❶ Forward propagation
 - ❷ Calcule a função de custo
 - ❸ Backward propagation
 - ❹ **Atualize parâmetros** (usando parâmetros e grads do backprop)
- ❸ Use os parâmetros treinados para prever as saídas

Acrescentamos agora as opções de otimizadores do passo de atualização de parâmetros:

```
if optimizer == "gd":
    parameters = update_parameters_with_gd(parameters, grads,
                                             learning_rate)
elif optimizer == "momentum":
    parameters, v = update_parameters_with_momentum(
        parameters, grads, v, beta, learning_rate)
elif optimizer == "adam":
    t = t + 1 # Adam counter
    parameters, v, s = update_parameters_with_adam(parameters,
                                                    grads, v, s, t, learning_rate, beta1, beta2, epsilon)
```

Já vimos os passos gerais de uma rede neural:

- ① Inicializar parâmetros / Definir hiperparâmetros
- ② Iterar por `num_iter`:
 - ① Forward propagation
 - ② Calcule a função de custo
 - ③ Backward propagation
 - ④ **Atualize parâmetros** (usando parâmetros e grads do backprop)
- ③ Use os parâmetros treinados para prever as saídas

Acrescentamos agora as opções de otimizadores do passo de atualização de parâmetros:

```
if optimizer == "gd":
    parameters = update_parameters_with_gd(parameters, grads,
                                           learning_rate)
elif optimizer == "momentum":
    parameters, v = update_parameters_with_momentum(
        parameters, grads, v, beta, learning_rate)
elif optimizer == "adam":
    t = t + 1 # Adam counter
    parameters, v, s = update_parameters_with_adam(parameters,
                                                    grads, v, s, t, learning_rate, beta1, beta2, epsilon)
)
```